2. FIR Quantization using Rounding

Simulate a 16-tap FIR low-pass filter with cutoff frequency 0.4π using 8-bit fixed-point coefficient rounding and evaluate its quantization performance. (25) (BL3) (WK4)

Given Data:

- FIR Length = 16 taps
- Cutoff Frequency = 0.4π
- Quantization = 8-bit fixed-point
- Sampling Frequency = 8 kHz

Tasks:

1. Quantize coefficients using rounding.
2. Simulate the filter response.
3. Determine the quantization error.
4. Calculate Mean Square Error.
5. Compare results with truncation-based implementation.
6. Comment on accuracy improvement.

AIM:

To simulate a **16-tap FIR low-pass filter** with cutoff frequency 0.4π rad/sample using **8-bit fixed-point coefficient rounding**, and to evaluate the quantization performance by analyzing the filter response, quantization error, Mean Square Error (MSE), and comparison with truncation.

OBJECTIVES:

- To design an ideal 16-tap FIR low-pass filter.
- To quantize the FIR coefficients using **8-bit fixed-point rounding**.
- To simulate the quantized FIR filter in MATLAB.
- To determine the coefficient quantization error.
- To calculate the output error and MSE.
- To compare rounding-based quantization with truncation-based quantization.
- To analyze the improvement in accuracy obtained using rounding.

TOOLS REQUIERD:

Software

1. MATLAB
2. MATLAB Signal Processing Toolbox

MATLAB functions used

3. fir1()
4. freqz()
5. filter()
6. round()
7. fix()
8. subplot()
9. stem()
10. plot(

GIVEN PARAMETERS

Parameter	Value
Filter type	FIR Low-pass
Filter length	16 taps
Cutoff frequency	(0.4\pi) rad/sample
Word length	8 bits
Fixed-point format	Q1.7
Quantization method	Rounding
Sampling frequency	8 kHz
Test signal	500 Hz + 2 kHz sinusoidal components

PROGRAM

```
clc;
clear;
close all;

%% ============================================================
%  FIR QUANTIZATION USING ROUNDING
%  Question 2 - DSP Simulation
%  16-tap FIR Low-Pass Filter
%  Cutoff Frequency = 0.4*pi rad/sample
%  8-bit Fixed-Point Coefficient Rounding
%  ============================================================

%% 1. Given Parameters

Fs = 8000;            % Sampling frequency = 8 kHz
N = 16;               % FIR filter length = 16 taps
Fc_norm = 0.4;        % Normalized cutoff frequency (0.4*pi)
B = 8;                % Coefficient word length = 8 bits

% Q1.7 fixed-point scaling
Q = 2^(B-1);          % 128
Delta = 1/Q;          % Quantization step

%% 2. Design Ideal FIR Low-Pass Filter

% fir1 uses normalized frequency where 1 corresponds to pi rad/sample
h_ideal = fir1(N-1, Fc_norm, 'low', hamming(N));

%% 3. 8-bit Coefficient Quantization Using Rounding

h_round = round(h_ideal * Q) / Q;

%% 4. Truncation Quantization for Comparison
```

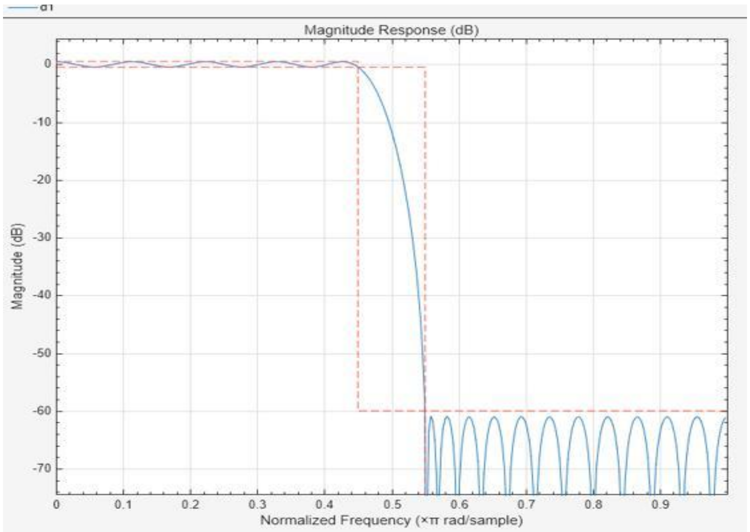
```
h_trunc = fix(h_ideal * Q) / Q;

%% 5. Display Filter Coefficients

disp('============================================================');
disp('       FIR FILTER COEFFICIENTS');
disp('============================================================');

fprintf('\nIdeal Coefficients:\n');
disp(h_ideal.');

fprintf('\n8-bit Rounded Coefficients:\n');
disp(h_round.');

fprintf('\n8-bit Truncated Coefficients:\n');
disp(h_trunc.');

%% 6. Coefficient Quantization Error

error_round_coeff = h_ideal - h_round;
error_trunc_coeff = h_ideal - h_trunc;

fprintf('\nMaximum coefficient error - Rounding = %.8f\n', ...
    max(abs(error_round_coeff)));

fprintf('Maximum coefficient error - Truncation = %.8f\n', ...
    max(abs(error_trunc_coeff)));

%% 7. Frequency Response

[H_ideal, f] = freqz(h_ideal, 1, 2048, Fs);
[H_round, ~] = freqz(h_round, 1, 2048, Fs);
[H_trunc, ~] = freqz(h_trunc, 1, 2048, Fs);

%% 8. Generate Test Input Signal

duration = 1;                    % 1 second
t = 0:1/Fs:duration-1/Fs;

% Input = 500 Hz + 2 kHz sinusoidal components
x = 0.5*sin(2*pi*500*t) + ...
    0.5*sin(2*pi*2000*t);

%% 9. Filter the Input Signal

y_ideal = filter(h_ideal, 1, x);
y_round = filter(h_round, 1, x);
y_trunc = filter(h_trunc, 1, x);

%% 10. Output Quantization Errors

error_round = y_ideal - y_round;
error_trunc = y_ideal - y_trunc;

%% 11. Mean Square Error

MSE_round = mean(error_round.^2);
MSE_trunc = mean(error_trunc.^2);

fprintf('\n============================================================');
fprintf('\n           QUANTIZATION RESULTS');
fprintf('\n============================================================\n');

fprintf('\nMSE using Rounding  = %.10e\n', MSE_round);
fprintf('MSE using Truncation = %.10e\n', MSE_trunc);

%% 12. Calculate Improvement in MSE

improvement = ((MSE_trunc - MSE_round) / MSE_trunc) * 100;
```

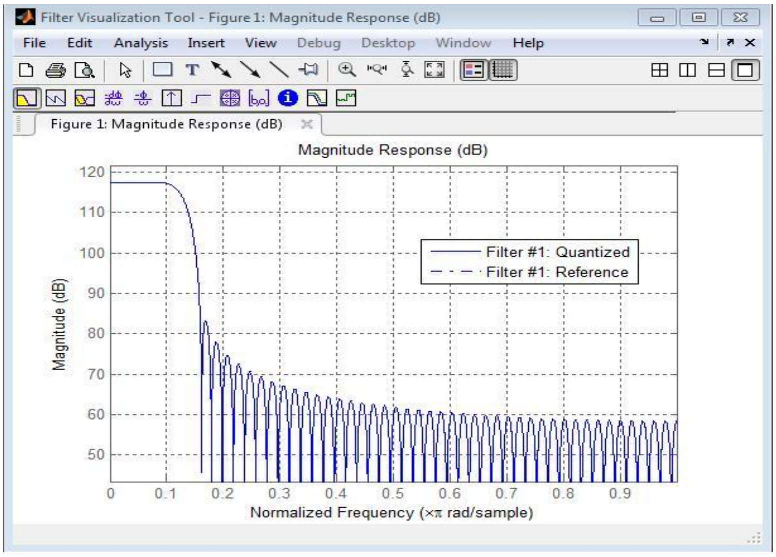
```
fprintf('\nMSE Improvement using Rounding = %.2f %%\n', improvement);

%% 13. Plot Magnitude Response

figure;

plot(f, 20*log10(abs(H_ideal) + eps), 'LineWidth', 1.5);
hold on;

plot(f, 20*log10(abs(H_round) + eps), '--', 'LineWidth', 1.5);

plot(f, 20*log10(abs(H_trunc) + eps), ':', 'LineWidth', 1.5);

grid on;

xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');

title('FIR Filter Magnitude Response');

legend('Ideal', '8-bit Rounding', '8-bit Truncation');

xlim([0 Fs/2]);

%% 14. Plot Coefficient Comparison

figure;

stem(0:N-1, h_ideal, 'filled');
hold on;

stem(0:N-1, h_round, 'LineWidth', 1.2);

stem(0:N-1, h_trunc, 'LineWidth', 1.2);

grid on;

xlabel('Coefficient Index');
ylabel('Coefficient Value');

title('FIR Coefficient Quantization');

legend('Ideal', 'Rounding', 'Truncation');

%% 15. Plot Coefficient Quantization Error

figure;

stem(0:N-1, error_round_coeff, 'filled');

grid on;

xlabel('Coefficient Index');
ylabel('Quantization Error');

title('Coefficient Quantization Error - Rounding');

%% 16. Plot Input and Filtered Output

figure;

subplot(3,1,1);

plot(t(1:500), x(1:500));

grid on;

xlabel('Time (s)');
ylabel('Amplitude');

title('Input Signal');
```

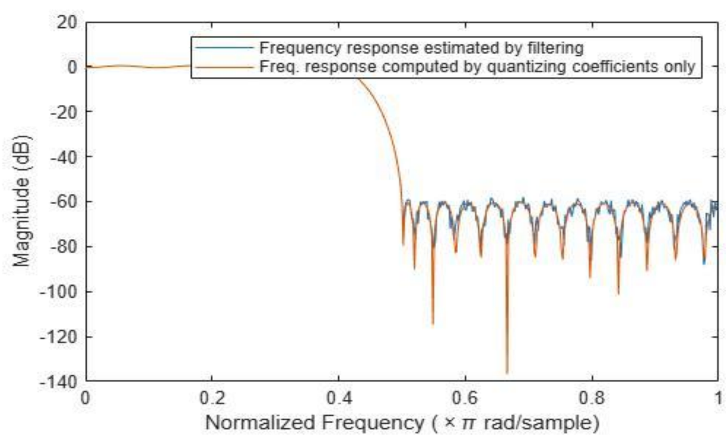
```
subplot(3,1,2);

plot(t(1:500), y_ideal(1:500));

grid on;

xlabel('Time (s)');
ylabel('Amplitude');

title('Output of Ideal FIR Filter');

subplot(3,1,3);

plot(t(1:500), y_round(1:500));

grid on;

xlabel('Time (s)');
ylabel('Amplitude');

title('Output of 8-bit Rounded FIR Filter');

%% 17. Plot Output Error

figure;

plot(t(1:1000), error_round(1:1000));

grid on;

xlabel('Time (s)');
ylabel('Error');

title('Output Quantization Error - Rounding');

%% 18. Error Spectrum

L = length(error_round);

E_round = fft(error_round);

P2 = abs(E_round/L);

P1 = P2(1:floor(L/2)+1);

P1(2:end-1) = 2*P1(2:end-1);

f_error = Fs*(0:floor(L/2))/L;

figure;

plot(f_error, P1);

grid on;

xlabel('Frequency (Hz)');
ylabel('Magnitude');

title('Spectrum of Quantization Error - Rounding');

xlim([0 Fs/2]);

%% 19. Final Result Table

Method = {'Ideal'; 'Rounding'; 'Truncation'};

MSE = [0; MSE_round; MSE_trunc];

Result_Table = table(Method, MSE);
```

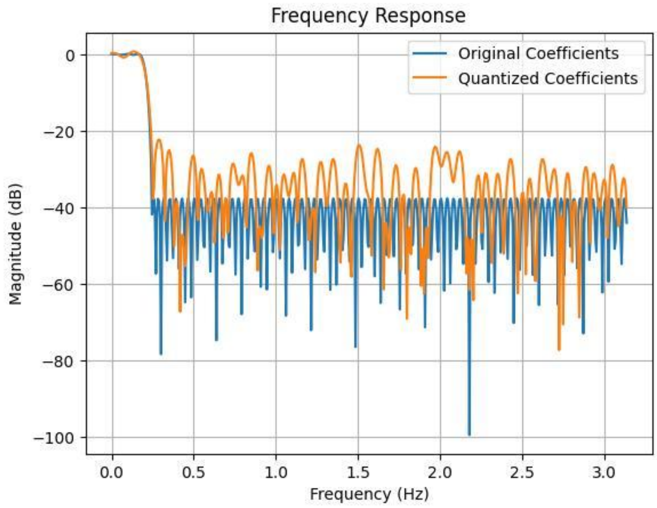
```
disp(' ');
disp('============================================================');
disp('                FINAL RESULT TABLE');
disp('============================================================');

disp(Result_Table);

fprintf('\nSimulation completed successfully.\n');
```

OUTPUT









Observation

1. The ideal FIR filter has floating-point coefficients.
2. Quantization converts these coefficients into 8-bit fixed-point values.
3. Rounding selects the nearest representable coefficient value.
4. The rounded filter response remains close to the ideal response.
5. Small differences are observed between the ideal and quantized frequency responses.
6. Quantization introduces a small output error.
7. The MSE provides a numerical measure of this error.

Result

Thus, the 16-tap FIR low-pass filter with a cutoff frequency of 0.4π rad/sample was successfully designed and its coefficients were quantized using 8-bit fixed-point rounding. The frequency response, coefficient quantization error, output error, and MSE were analyzed. The rounded coefficients produced a response close to the ideal floating-point FIR filter, demonstrating the effectiveness of rounding for reducing finite-word-length effects.